Due: Thursday, September 17, 2026 11:59pm

Submit on GL.

---

1 Leap Year

For this question, you will be graded on the *good use* of guards.

Write a Haskell function `isLeapYear` that takes an `Int` value `n` and returns `True` when `n` is a leap year — i.e., the month of February has 29 days that year. If year `n` is not a leap year, your function should return `False`.

The rules leap year are: `n` is a leap year if `n` is divisible by 4 except when it is divisible by 100 unless it is also divisible by 400. So, 2024 (divisible by 4) was a leap year, but 2021 and 2022 (not divisible by 4) were not. Neither will 2100 (divisible by 100). However, the year 2000 (divisible by 400) was a leap year and so will 2400.

```
isLeapYear :: Int -> Bool
```

For a 10% bonus, make your guards as compact as possible. *Hint:* order matters.

Place your `isLeapYear` function in a file named `hw03_1.hs` and export this function in the module `HW03_1`. Include your name and userid as well. Submit your `hw03_1.hs` file on GL. The top of your file should look like:

```
module HW03_1 where
-- Ugo Boncompagni (pgreg13)
```

Your program should compile and run with `main03_1.hs` which can be found in the usual place on the GL filesystem.

```
linux3% ghc main03_1.hs hw03_1.hs -o main03_1.out
[1 of 2] Compiling HW03_1           ( hw03_1.hs, hw03_1.o )
[2 of 2] Compiling Main             ( main03_1.hs, main03_1.o )
Linking main03_1.out ...
linux3% ./main03_1.out

Testing isLeapYear...
2024 was a leap year: True
2022 was a leap year: False
2100 will be a leap year: False
2000 was a leap year: True
Done.
linux3%
```

2 Add Comma

Making good use of pattern matching, guards and recursion, write a Haskell function named `addcomma` that takes a `String` parameter `str` and inserts commas **between** the characters. For example, in `ghci` you should see:

```
ghci> addcomma "abc"
"a,b,c"
ghci> addcomma "b"
"b"
ghci> addcomma "abracadabra"
"a,b,r,a,c,a,d,a,b,r,a"
```

Do note that we do not want to insert a comma after the last character of the string, so choose your base cases carefully. This time your function should have type:

```
addcomma :: String -> String
```

Place your `addcomma` function in a file named `hw03_2.hs` and export this function in the module `HW03_2`. Include your name and userid as well. Submit your `hw03_2.hs` file on GL. The top of your file should look like:

```
module HW03_2 where
-- Ezra Koenig (oxcomma1)
```

Finally, your program should compile and run with `main03_2.hs` which can be found in the usual place on the GL filesystem.

```
linux3% ghc main03_2.hs hw03_2.hs -o main03_2.out
[1 of 2] Compiling HW03_2           ( hw03_2.hs, hw03_2.o )
[2 of 2] Compiling Main             ( main03_2.hs, main03_2.o )
Linking main03_2.out ...
linux3% ./main03_2.out

Testing addcomma:
Add commas to "abcdef":
a,b,c,d,e,f
Add commas to "":

Add commas to "x":
x
Add commas to "Vampire Weekend":
V,a,m,p,i,r,e, ,W,e,e,k,e,n,d
linux3%
```

3 Remove Max

Note: For this question, you should write all of the code yourself and not rely on any Haskell library functions other than simple functions like list manipulation and comparison.

Making good use of recursion (e.g., not list comprehension), define a Haskell function `rmax` removes all occurrences of the largest item in the list. Your `rmax` function should have type

```
rmax :: Ord a => [a] -> [a]
```

Given the empty list, `rmax` should return the empty list. You will likely need to define helper functions. It is OK, if your wrapper function is not recursive and only the helper functions are recursive.

For a 10% bonus, make your `rmax` function work in one pass. If for example, you find the maximum item recursively and give this to a recursive function to remove from the list, then that is two passes. Using a library function to find the maximum and/or list comprehension to remove the maximum does not count. You must find and remove the maximum in one pass. *Hint:* find the maximum item on the way down and remove each occurrence on the way back up.

Name your file `hw03_3.hs` and export the `rmax` function in the module `HW03_3`. Include your name and userid as well. The top of your file should look like:

```
module HW03_3 where
-- Matt Frewer (headrm1)
```

Your `hw03_3.hs` file should compile with `main03_3.hs` which can be found on GL at the usual place.

```
linux3% ghc main03_3.hs hw03_3.hs -o main03_3.out
[1 of 2] Compiling HW03_3           ( hw03_3.hs, hw03_3.o )
[2 of 2] Compiling Main             ( main03_3.hs, main03_3.o )
Linking main03_3.out ...
linux3% ./main03_3.out
[3,2,8,7,1,6]
[]
[9,4,13,2]
[1001,9,13]
[]
["Fido","Ginger"]
linux3%
```

4 List Comprehension: Pythagorean Triples

Note: For this question, you will be graded on good use of list comprehension.

A Pythagorean triple (a, b, c) is a triple of three positive integers such that $a^2 + b^2 = c^2$. Write a Haskell function `ptriples` that takes an `Int` value n and returns a list of all Pythagorean triples where a, b and c are less than or equal to n. For example:

```
> ptriples 13
[(3,4,5),(4,3,5),(5,12,13),(6,8,10),(8,6,10),(12,5,13)]
```

Note: for this question, you must use list comprehension to implement your function.

Put your `ptriples` function in a file named `hw03_4.hs` and export these functions in the module `HW03_4`. Include your name and userid as well. Submit your `hw03_4.hs` on GL. The top of your file should look like:

```
module HW03_4 where
-- Alois Lutz (fsjump3)
```

Your `hw03_4.hs` file should compile with `main03_4.hs` using `ghc`. You can find `main03_4.hs` in the usual place on GL. Your output might look like:

```
linux3% ghc main03_4.hs hw03_4.hs -o main03_4.out
[1 of 2] Compiling HW03_4           ( hw03_4.hs, hw03_4.o )
[2 of 2] Compiling Main             ( main03_4.hs, main03_4.o )
Linking main03_4.out ...
linux3% ./main03_4.out
[(3,4,5),(4,3,5),(5,12,13),(6,8,10),(8,6,10),(12,5,13)]

[(3,4,5),(4,3,5),(5,12,13),(6,8,10),(7,24,25),(8,6,10),
(8,15,17),(9,12,15),(12,5,13),(12,9,15),(12,16,20),(15,8,17),
(15,20,25),(16,12,20),(20,15,25),(24,7,25)]
linux3%
```